

ESP32-DevKitC (38-pin USB-C)

Project Reference — ESP-WROOM-32 · Wi-Fi · Bluetooth · 32 GPIOs

1. Overview

This is a 38-pin ESP32-DevKitC-style development board built around the **ESP-WROOM-32** module. The ESP-WROOM-32 contains an Espressif ESP32 dual-core 32-bit Tensilica LX6 microprocessor running at up to 240 MHz, with integrated Wi-Fi (802.11 b/g/n) and Bluetooth (Classic + BLE) — all in a shielded module on top of the board.

The development board adds a **USB-C connector**, a **CP2102 USB-UART bridge** (the listing's "CP2012" is a typo), a 3.3 V LDO regulator, BOOT and EN buttons for flashing, and breaks out 38 pins to two rows of 0.1" headers. The narrower 38-pin layout fits across a standard breadboard with one row of holes free on each side for jumper wires — the main reason this version is preferred over wider ESP32 boards.

Key capabilities:

- Dual-core Xtensa LX6 @ up to 240 MHz, 520 KB SRAM, 4 MB flash
- Wi-Fi 802.11 b/g/n (2.4 GHz) — STA, AP, and STA+AP modes
- Bluetooth 4.2 — Classic Bluetooth and Bluetooth Low Energy (BLE)
- 32 usable GPIO pads broken out (some input-only, some boot-strap)
- 18 channels of 12-bit ADC, 2 channels of 8-bit DAC
- 10 capacitive-touch GPIO pins
- 3 × UART, 2 × I²C, 3 × SPI, 2 × I²S, 16 × PWM channels, CAN, Ethernet MAC
- Hardware crypto accelerator (AES, SHA, RSA, ECC, RNG)
- USB-C for power and serial programming via CP2102
- Programmable from Arduino IDE, ESP-IDF, MicroPython, PlatformIO, and more

2. Specifications

Parameter	Value
Module	ESP-WROOM-32 (Espressif)
MCU	ESP32 dual-core Xtensa LX6, up to 240 MHz
Flash memory	4 MB SPI flash (typical)
SRAM	520 KB internal
Wi-Fi	802.11 b/g/n, 2.4 GHz, STA / AP / STA+AP
Bluetooth	v4.2 BR/EDR + BLE
USB connector	USB Type-C (data + power)
USB-UART bridge	CP2102 (Silicon Labs)
Logic level	3.3 V (5 V tolerant on most pins, see notes)
Operating voltage (VIN)	5 V via USB-C, or 5 V on the VIN/5V pin
3.3 V regulator	Onboard LDO (~600 mA typical)
Operating current (active)	~80–260 mA depending on Wi-Fi activity
Deep sleep current	~10 μ A
GPIO header	2 $\times$ 19-pin (38 total), 0.1" / 2.54 mm pitch
Board dimensions	$\approx$ 51 $\times$ 25 mm (narrow profile, breadboard-friendly)
Operating temperature	–40 $^{\circ}$ C to +85 $^{\circ}$ C (chip rating)

Warning: ESP32 GPIOs are **3.3 V logic only**. Connecting 5 V signals directly to a GPIO (e.g. a 5 V sensor's data line) can damage the chip. Use a level shifter or voltage divider for 5 V peripherals. The 5V/VIN pin *accepts* 5 V but only as a power input — never as a signal.

3. GPIO Pinout & Functions

The 38-pin board layout follows the standard ESP-WROOM-32 reference. Pin numbers below match the physical board layout, counted from the top of each row (USB connector at the bottom, antenna at the top). Each GPIO can be remapped in software for most peripherals — the labels below are the **default / Arduino-convention** functions.

Left side header (component side up, USB at bottom)

Pin	Label	GPIO	Default function / notes
1	3V3	—	3.3 V output from onboard LDO (~600 mA max)
2	EN	—	Chip enable (active HIGH). Pull LOW = reset.
3	VP	GPIO36	Input only. ADC1_CH0. No internal pull-up/pull-down.
4	VN	GPIO39	Input only. ADC1_CH3. No internal pull-up/pull-down.
5	D34	GPIO34	Input only. ADC1_CH6.
6	D35	GPIO35	Input only. ADC1_CH7.
7	D32	GPIO32	ADC1_CH4, Touch9, XTAL_32K_P
8	D33	GPIO33	ADC1_CH5, Touch8, XTAL_32K_N
9	D25	GPIO25	DAC_1, ADC2_CH8
10	D26	GPIO26	DAC_2, ADC2_CH9
11	D27	GPIO27	ADC2_CH7, Touch7
12	D14	GPIO14	HSPI SCK, ADC2_CH6, Touch6 — strapping pin
13	D12	GPIO12	HSPI MISO, ADC2_CH5, Touch5 — must be LOW at boot
14	D13	GPIO13	HSPI MOSI, ADC2_CH4, Touch4
15	GND	—	Ground
16	VIN/5V	—	5 V input/output (connected to USB 5 V rail)

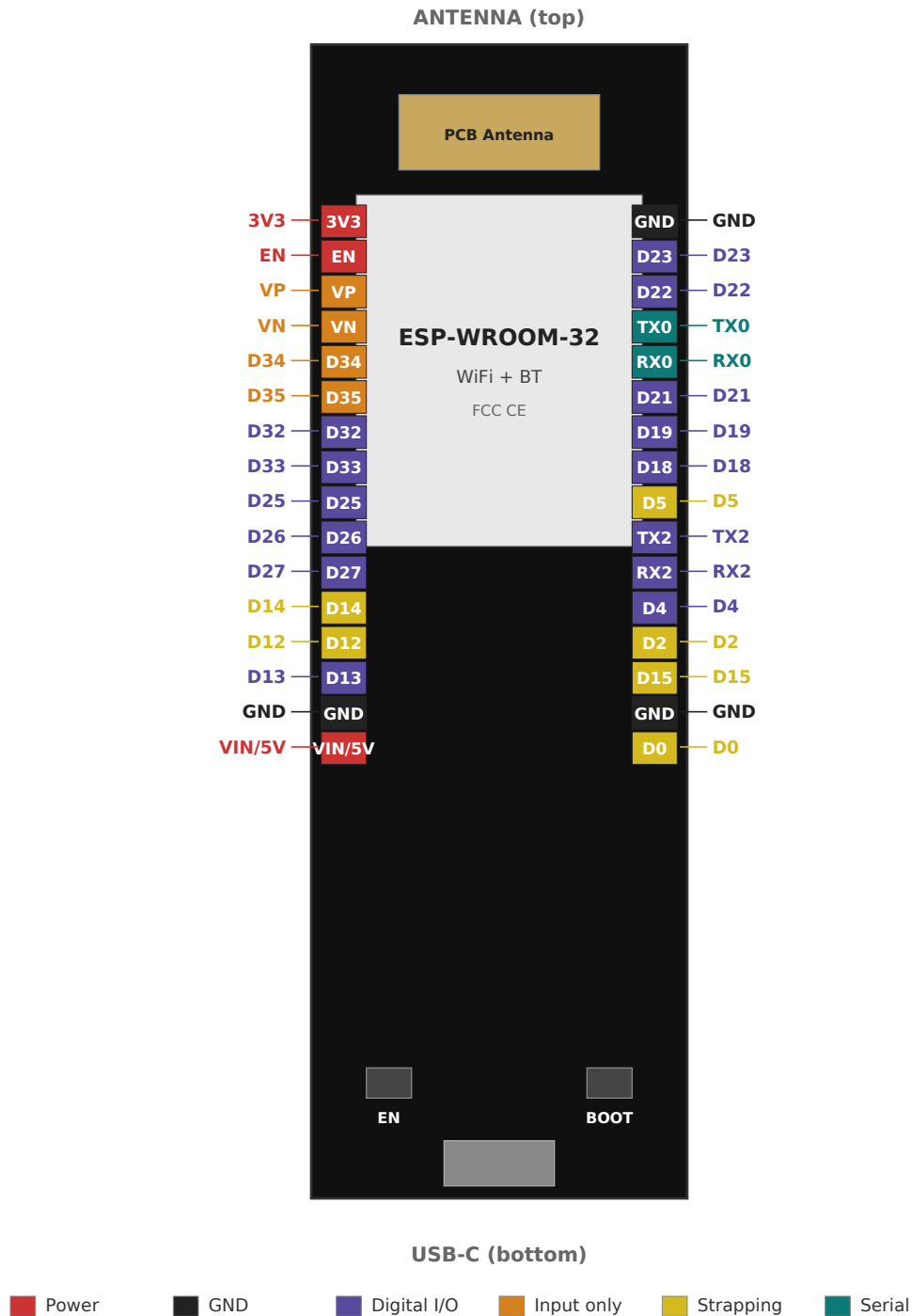
Right side header (component side up, USB at bottom)

Pin	Label	GPIO	Default function / notes
1	GND	—	Ground
2	D23	GPIO23	VSPI MOSI (default Arduino SPI MOSI)
3	D22	GPIO22	I ² C SCL (default Arduino Wire SCL)
4	TX0	GPIO1	UART0 TX — connected to CP2102 (USB serial)
5	RX0	GPIO3	UART0 RX — connected to CP2102 (USB serial)
6	D21	GPIO21	I ² C SDA (default Arduino Wire SDA)
7	D19	GPIO19	VSPI MISO (default Arduino SPI MISO)
8	D18	GPIO18	VSPI SCK (default Arduino SPI SCK)
9	D5	GPIO5	VSPI CS / SS — strapping pin (HIGH at boot)
10	TX2	GPIO17	UART2 TX
11	RX2	GPIO16	UART2 RX
12	D4	GPIO4	ADC2_CH0, Touch0
13	D2	GPIO2	Onboard blue LED, ADC2_CH2, Touch2 — strapping pin
14	D15	GPIO15	ADC2_CH3, Touch3 — strapping pin (HIGH at boot)
15	GND	—	Ground
16	D0	GPIO0	BOOT button — strapping pin (LOW at boot = flash mode)

Note on pin numbers: the physical pin numbers above (1-16 per side) refer to position on the header. The **GPIO number** is what you use in code (e.g. `digitalWrite(23, HIGH)`). Some pins like 3V3 and GND have no GPIO assignment.

4. Board Pinout Diagram

Top-down view of the board with the USB-C connector at the bottom and the antenna at the top. Color coding indicates pin category: power (red/black), digital I/O (purple), input-only (orange), strapping pins (yellow — use with care during boot), serial (teal).



Component side view. Pin order matches the physical board with USB-C facing down.

5. Pin Selection Guide (Which GPIOs to Use)

Not all GPIOs are equally safe to use. Some affect boot behavior, some are input-only, some are tied to internal flash. Use the categories below as a starting point.

Category	GPIOs	Notes
SAFE — use freely	4, 5, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33	Full-featured digital I/O. No boot-time gotchas.
CAUTION — strapping pins	0, 2, 5, 12, 15	Affect boot mode. Don't pull them in unexpected directions during reset. GPIO0 = BOOT button. GPIO12 must be LOW at boot or flash voltage selection breaks.
CAUTION — input only	34, 35, 36 (VP), 39 (VN)	Cannot drive outputs. No internal pull-up or pull-down. Great for pure sensor inputs.
DON'T USE — internal flash	6, 7, 8, 9, 10, 11	Connected to onboard SPI flash. Touching them will crash the chip. They are not broken out on this board.
NOTE — Wi-Fi conflict	ADC2 channels: 0, 2, 4, 12, 13, 14, 15, 25, 26, 27	ADC2 cannot be read while Wi-Fi is active. Use ADC1 pins (32, 33, 34, 35, 36, 39) for analog reads when Wi-Fi is on.

Best practice: for I²C use the defaults (SDA=21, SCL=22). For SPI use VSPI defaults (MOSI=23, MISO=19, SCK=18, CS=5). For analog reads with Wi-Fi on, use GPIO 32, 33, 34, 35, 36, or 39. For touch sensing, use any of the touch-capable pins (Touch0–Touch9 on GPIO 4, 0, 2, 15, 13, 12, 14, 27, 33, 32).

6. Power Options

There are several ways to power the board, each with trade-offs. Pick one — never apply power from two sources at the same time.

Method	Pin	Notes
USB-C cable (typical)	USB-C connector	5 V from USB. Easiest. Powers everything through the onboard regulator. Also gives you serial console + flashing.
External 5 V supply	VIN / 5V pin	Feed regulated 5 V here (4.5–5.5 V). Goes through the onboard 3.3 V LDO. Common when the board is in a project enclosure with no USB.
External 3.3 V supply	3V3 pin	Feed clean 3.3 V directly here. Bypasses the onboard regulator. Useful for very low-power battery designs.
LiPo battery (with shield)	Add-on board	Bare ESP32-DevKitC has no battery management. For battery projects use a TP4056 charger + a 3.3 V buck-boost (like the TPS63070 module).

Current draw: the ESP32 pulls anywhere from 10 μ A (deep sleep) to ~260 mA (peak Wi-Fi TX). If your supply or USB cable is marginal, you'll see brown-out resets — especially when Wi-Fi connects. Use a quality USB cable and a 1 A or higher supply. Add a 100–470 μ F electrolytic between 5V and GND for long power leads.

7. Flashing & Programming

The board comes with a CP2102 USB-UART bridge that handles flashing automatically. On Linux/macOS drivers are built-in; on Windows install the Silicon Labs CP210x VCP driver if needed.

7.1 Arduino IDE setup

File → Preferences → Additional Boards Manager URLs:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

Tools → Board → Boards Manager → search "esp32" → install "esp32 by Espressif Systems"

Tools → Board → ESP32 Arduino → "ESP32 Dev Module" (or "DOIT ESP32 DEVKIT V1")
Tools → Port → /dev/ttyUSB0 (Linux) or COMx (Windows) or /dev/cu.SLAB_USBtoUART (Mac)
Tools → Upload Speed → 921600 (or 115200 if upload fails)

7.2 Blink example

```
// The onboard blue LED is on GPIO 2
const int LED = 2;

void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(115200);
  Serial.println("ESP32 starting up");
}

void loop() {
  digitalWrite(LED, HIGH);
  delay(500);
  digitalWrite(LED, LOW);
  delay(500);
  Serial.println("blink");
}
```

7.3 Wi-Fi connection example

```
#include <WiFi.h>

const char* ssid      = "YourWiFi";
const char* password = "YourPassword";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected. IP: ");
  Serial.println(WiFi.localIP());
}

void loop() { }
```


8. Default Peripheral Pins (Arduino conventions)

The ESP32 multiplexes peripherals — almost any GPIO can host any function. But these are the defaults the Arduino-ESP32 core uses unless you override them in code.

Peripheral	Signal	Default GPIO
I ² C (Wire)	SDA	GPIO 21
I ² C (Wire)	SCL	GPIO 22
SPI (VSPI)	MOSI	GPIO 23
SPI (VSPI)	MISO	GPIO 19
SPI (VSPI)	SCK	GPIO 18
SPI (VSPI)	CS / SS	GPIO 5
SPI (HSPI alt)	MOSI	GPIO 13
SPI (HSPI alt)	MISO	GPIO 12
SPI (HSPI alt)	SCK	GPIO 14
SPI (HSPI alt)	CS / SS	GPIO 15
UART0 (USB)	TX / RX	GPIO 1 / GPIO 3
UART2	TX / RX	GPIO 17 / GPIO 16
DAC1	Analog out	GPIO 25
DAC2	Analog out	GPIO 26
Onboard LED	Active HIGH	GPIO 2
BOOT button	Active LOW	GPIO 0

9. Troubleshooting

Upload fails: "Failed to connect / Timed out waiting for packet header": Hold the BOOT button while clicking Upload, release after "Connecting..." appears. Some boards with USB-C don't auto-reset reliably. Try lowering upload speed to 115200.

Board not detected on USB: Install the CP2102 driver (Silicon Labs CP210x VCP). Try a different USB-C cable — many cheap cables are charge-only with no data lines.

Random reboots / brownout when Wi-Fi connects: Power supply or cable can't provide the ~260 mA peak. Use a 1 A+ supply, a better cable, and add a 100–470 μ F capacitor across 5V and GND.

Serial Monitor shows garbage characters: Wrong baud rate. ESP32 boots at 115200 by default. Set Serial Monitor to match `Serial.begin()` in your sketch.

analogRead always returns 4095 or 0 with Wi-Fi on: You're using an ADC2 pin (GPIO 0, 2, 4, 12–15, 25–27). ADC2 conflicts with Wi-Fi. Switch to an ADC1 pin (GPIO 32–39).

Sketch boots into flash mode every time: Something is pulling GPIO0 LOW at boot. Check that nothing on your circuit is driving GPIO0 low during reset. Also check GPIO12 — if pulled HIGH externally, the chip selects 1.8 V flash and won't boot.

Onboard LED doesn't blink with the standard Arduino "Blink" example: The default Blink uses `LED_BUILTIN` which may not be defined for your board variant. Use `const int LED = 2;` explicitly — the blue LED is on GPIO 2.

Board gets very hot: Check for shorts on your circuit. The ESP32 itself runs warm under heavy Wi-Fi but should not be painful to touch. If the regulator IC near the USB connector is the hot spot, your 3V3 rail is pulling too much current.

Wi-Fi connects but is very slow / drops out: The PCB antenna is at the top of the board. Don't put metal objects near it, and don't mount the board directly on a metal enclosure without offset. Range is meaningfully reduced if the antenna edge is over a ground plane.

10. References & Further Reading

- **Espressif — ESP32-DevKitC official getting-started guide**

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html>

- **Espressif — ESP32 Technical Reference Manual (PDF)**

https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

- **Espressif — ESP-WROOM-32 datasheet (PDF)**

https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

- **Arduino-ESP32 core (GitHub)**

<https://github.com/espressif/arduino-esp32>

- **Random Nerd Tutorials — ESP32 Pinout Reference**

<https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>

- **Last Minute Engineers — ESP32 Pinout Guide**

<https://lastminuteengineers.com/esp32-pinout-reference/>

- **Silicon Labs CP210x VCP driver download**

<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>

- **PlatformIO — ESP32 platform docs**

<https://docs.platformio.org/en/latest/platforms/espressif32.html>

Reference compiled from Espressif's official documentation and community sources. GPIO numbers refer to the underlying ESP32 chip — physical pin positions match the standard ESP32-DevKitC 38-pin layout. Always verify silkscreen labels on your specific board revision.